

Leveraging Large Language Models for Classification of Grammaticalization Levels in German Prepositions

Prasanna Bhat, Divyansh Kaushik, Luca Burghard,
University of Technology Nuremberg, Germany

{prasanna.bhat, divyansh.kaushik, luca.burghard}@utn.de

Abstract

Grammaticalization is the gradual shift of lexical items into grammatical functions. Previous studies have estimated the extent of grammaticalization using corpus measures (e.g. frequency and contextual diversity). Here, we predict levels directly using large language models (LLMs), evaluating zero-shot and few-shot prompts. Across settings, LLMs outperform corpus baselines (frequency and syntactic context adversity), with optimized few-shot prompting achieving the best results. These results suggest that LLMs can provide accurate, context-aware annotations and offer a scalable approach to automated grammaticalization analysis.

1 Introduction

Grammaticalization has been a topic of linguistic interest for a long time, and it is difficult to pinpoint its exact origins in linguistic research. In general terms, it is the process by which a content word, such as a noun, verb, or adjective, gradually evolves from its original lexical form (its core, dictionary meaning) into a grammatical element, such as a preposition, often losing its specific meaning in the process and gaining more abstract, functional roles. This shift typically involves moving from a more concrete meaning to a more abstract grammatical role, sometimes as part of a larger fixed expression (Hopper and Traugott, 2003). Understanding and tracking this process is important for studying language change because it provides valuable insights into how grammar develops and evolves over time. An example of this can be seen in the development of the old English construction *pa hwile be* ('that time that'), which over time grammaticalized into the modern English *while* (Hopper and Traugott, 2003).

We are interested in predicting grammaticalization levels because manual annotation by experts is costly and time-consuming, which makes large-scale studies impractical. If we can automatically

determine these levels, we can analyze vast datasets much more efficiently and investigate how grammaticalization changes over time.

Previous studies have tackled this issue by using statistical, corpus-based metrics such as word frequency, distributional entropy, and collocation measures (Schlechtweg and Walde, 2018). While these methods have shown promise, their performance has been far from perfect. LLMs have not yet been applied to this task, likely because earlier models lacked the necessary power to handle the complexity of grammaticalization. In this study, we explore the potential of modern LLMs to predict grammaticalization levels. We experiment with various techniques, such as few-shot prompting and providing explicit definitions of the levels, to ascertain whether they outperform traditional metrics-based methods.

2 Related Work

2.1 Grammaticalization Level Prediction

The first approach to solve the task was to split it into four levels. Schlechtweg and Walde (2018) were the first to introduce these specific four levels, using definitions extracted from Meola (2012).

While Schlechtweg and Walde (2018) do not provide explicit definitions for each level, it formed the basis for our own definitions, which we established by consulting two references by Di Meola, that Schlechtweg and Walde (2018) used to find their definitions of the 4 levels (Di Meola, 2014; Meola, 2012). These definitions range from Level 1, representing the lowest degree of grammaticalization, to Level 4, representing the highest. The exact definitions can be found in subsection 3.1.

In their study, Schlechtweg and Walde (2018) extracted a test set of 206 German prepositions from Meola (2012) annotated by humans with the four levels and explored the predictive power of statistical corpus-based measures. Using the *SdeWaC*

corpus Baroni et al. (2009), they calculated metrics such as word entropy, word frequency and evaluated the correlation of these metrics to the grammaticalization level using Spearman’s rank correlation and Average Precision. They found that frequency and context type counts outperformed entropy, but all three measures showed very similar and moderate correlations with the annotated levels. There were multiple other approaches to calculate some kind of level of grammaticalization based on corpus-wide metrics like phonological length, collocational diversity, colligate diversity, and dispersion. (Correia Saavedra, 2019). Further, Hu et al. (2025) have shown that grammatical function words tend to have higher entropy in general.

Building on this, our work replicates the central task of predicting grammaticalization levels for the same test set as Schlechtweg and Walde (2018), but introduces a key difference: we employ LLMs rather than relying solely on statistical metrics. This allows us to test whether LLMs can surpass these baselines by capturing deeper semantic and syntactic cues related to grammaticalization that traditional distributional measures may overlook.

2.2 Prompting and LLM-Based Strategies

Prompt-based classification enables language models to handle subjective labeling tasks using natural language instructions. Inspired by Loke et al. (2025), who used LLMs to approximate human similarity judgments, we apply the same strategy to rating grammaticalization on a four-level scale. Instead of relying only on bare label names, we provide short natural-language definitions, giving the model more context. Prior work supports this approach: in scientific and financial domains, Walshe et al. (2025) shows that label descriptions help align predictions with human judgments, and Munker et al. (2024) demonstrates similar gains in German tweet labeling.

In few-shot prompting, the method to present examples deeply impacts performance. One critical factor is the order of examples, which can swing accuracy from near chance to almost fully supervised levels. Lu et al. (2022) highlighted this sensitivity and showed that simply rearranging examples matters. Zhao et al. (2021) traced instability in few-shot results largely to inherent model biases, including favoring majority labels, recency effects, and more common tokens, which they addressed

through class-balanced labeled examples.

Even though manual prompt crafting is very effective, it can be slow, inconsistent, or prone to errors. Finding the right mix of format, examples, and instructions often takes endless trial and error. *Declarative Self-improving Python* (DSPy) tackles this by treating prompt design like programming, where one builds modular LM ‘programs’ whose instructions and few-shot examples can be optimized together (Khattab et al., 2023). DSPy’s *Multiprompt Instruction Proposal Optimizer Version 2* (MIPROv2) uses Bayesian search to propose and refine both instructions and bootstrapped examples, while figuring out which changes truly improve overall performance (Opsahl-Ong et al., 2024). Evaluating in mini-batches makes the most of limited optimization runs and consistently delivers bigger gains when tuning instructions and examples jointly.

2.3 Data Leakage and Validation Concerns

Given the grammaticalization benchmark is public, evaluated LLMs may have seen it or its close variants during training. Such contamination can inflate scores and mislead conclusions. We therefore run leakage checks to detect both direct memorization and indirect exposure before the main experiments.

Detection of contamination relies on black-box probes: *reconstruction* (LM Contamination Index) that prompts regeneration of dataset items (Balloccu et al., 2024), *completion overlap* similarity between continuations and references (Golchin and Surdeanu, 2024), and *extraction* via likelihood-ratio ranking; these also motivate de-duplication and differentially private training (Carlini et al., 2021). For closed-source LLMs with opaque, web-scale training, undisclosed corpora/preprocessing further heighten contamination risk.

Early results showed GPT-2 can reproduce hundreds of training samples verbatim, including PII (Carlini et al., 2021); subsequent work extended this to production-scale models, with larger models and targeted prompts increasing memorization (Carlini et al., 2023). Contamination is both *direct* (benchmark items or near-duplicates in pre-training) and *indirect* (evaluation data later used in updates): a review found ~42% of 255 papers leaked ~4.7M samples across 263 datasets (e.g., Semeval2016 Task 6, SAMSum, MultiWOZ) (Balloccu et al., 2024). Because each method has limits

Expression (German)	Translation (English)	Level
wegen	because of	4
im Abstand	at a distance	3
am Rand	at the edge	2
auf dem Boden	on the ground	1
...	...	...

Table 1: Examples from the ground-truth dataset with added german translations.

(filters, paraphrase/truncation), we triangulate evidence with lightweight probes; partial recall, real-fake gaps, half set context and report setup/results in the Methods and Experiments sections.

3 Data

3.1 Grammaticalization Data

The ground truth dataset¹ originates from the work of Schlechtweg and Walde (2018). It contains **206 German Expressions**, each annotated with a grammaticalization score between 1 and 4 annotated by a deep linguistic analysis. Definitions for each level extracted that we extracted from Meola (2012):

Level 1: Prepositional phrases at this level have a very low degree of grammaticalization and always consist of three words. Either the structure is *preposition + NP + preposition*. A second possible structure is *preposition + NP*, in which an article always appears.

Level 2: Formations at this level show a slightly higher degree of grammaticalization, as they can occur both with and without an article. In some cases, the use with an article still outweighs the use without one, while in other formations, the share of occurrences without an autonomous article already dominates.

Level 3: Level 3 describes a state in which grammaticalization is already well advanced. The central feature of this stage is that the expressions (prepositional phrases) in the corpus examined were not assigned an attributive extension. This means that a descriptive adjective can no longer be inserted between the components of the phrase.

Level 4: Prepositions with the form of a function word have the highest degree of grammaticalization to be recorded, and the grammaticalization process is for the most part complete. Due to phonological and/or semantic erosion, the original structure is no longer recognizable, and these forms thus receive the status of “pure” function words.

This dataset serves as our ground truth for evaluation, against which we compare both metric-based and LLM-based predictions. The structure of the ground truth dataset can be seen in Table 1.

The distribution of expressions per grammaticalization levels is plotted in Figure 1.

¹<https://evolang.org/torun/proceedings/SIMaterial/SIPaper169.zip>

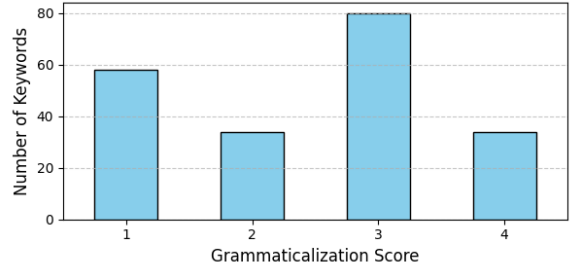


Figure 1: Expression distribution across grammaticalization levels in the ground truth dataset.

Dataset	expressions
Dev Data 1 (Few-shot Prompting)	32
Dev Data 2 (Development Testing)	75
Test Data (Final Evaluation)	99

Table 2: Three-way split of the ground truth dataset.

3.2 Corpus for Metric Calculations

For computing statistical features like frequency or entropy, we used the same dataset as the Schlechtweg and Walde (2018): **SdeWaC corpus (German version, 0.88 billion words)**, a large-scale web corpus with POS-tagging (Baroni et al., 2009). The exact version used was **sdewac-v3.txt**.

Corpus Statistics:

- File size: ~19 GB
- Tokens: ~884 million
- Sentences: ~43 million

3.3 Data Splits

To prevent data leakage and ensure fair evaluation, the ground truth dataset was randomly divided into three non-overlapping subsets. *Development Data 1* contains a small set of expressions for few-shot prompting. *Development Data 2* is used for development testing and hyperparameter tuning, while the Test Data is reserved exclusively for final evaluation. The split ensures that expressions appearing in one set never appear in another. The amount of expressions per split is shown in Table 2.

In addition to the main splits, we also created an auxiliary dataset containing example sentences for each expression. Specifically, we extracted 10 sentences from the SdeWaC corpus that for each expression of our ground-truth dataset contained the given expression. The intention behind this dataset was to provide contextual examples that could be used for LLM-based predictions; for instance, by presenting an expression alongside sev-

eral sentences in which it appears and asking the model to predict its grammaticalization level.

3.4 Leakage Datasets

c4de. Diagnostic split from German C4 (Common Crawl), using the gold-set expression inventory to test pretraining exposure.

wiki. Diagnostic split from German Wikipedia, probing exposure via encyclopedic text.

Rationale. C4 and Wikipedia are common pre-training sources and thus plausible leakage points. These splits are used *only* for leakage diagnosis (partial recall, real-fake gap, half-set), not for tuning or final accuracy.

4 Method

4.1 Baseline Metric

POS-tags enabled us to analyze the syntactic context of each expression, e.g., computing the number of *distinct POS tags* occurring in surrounding positions, which we treat as a variation of entropy and a potential predictor of grammaticalization.

The corpus was processed to extract contextual and statistical features for each expression in the ground truth set. More on the chosen metrics we calculated from the dataset in [subsection 4.1](#).

We re-computed the baseline metrics from [Schlechtweg and Walde \(2018\)](#) original paper on the corpus and extended them with additional metrics that could correlate with grammaticalization levels. All code can be found in the our GitHub ²

Metrics calculated:

- **Frequency** — Total occurrences of all variants of the keyword in the full corpus.
- **Average character count** — Mean character length across all keyword variants.
- **Number of distinct neighbors** — Unique tokens occurring immediately before or after the keyword.
- **Syntactic context adversity** — Total number of unique part-of-speech tags found immediately before and after the keyword (Similar to *Number of distinct neighbors* but with POS-Tags).
- **Context entropy** — Shannon entropy of the surrounding context token distribution, reflecting lexical diversity ([Torres-García et al., 2022](#)).

²github.com/CastIehard/LLM-4-gram-lvl-pred

- **Collocation strength (weighted PMI)** — Frequency-weighted sum of pointwise mutual information values for keyword-context bi-grams.

4.2 Prompt Design Strategy

We adopt a dual strategy for prompt design: one is the human-curated prompt engineering, and the other one is automated prompt program synthesis using DSPy.

Human-Curated Prompting

Each prompt first assigns the model the role of a linguist specialized in German grammaticalization, priming it with appropriate expertise. This is followed by the core task instruction, which clearly states the classification objective. Finally, the model is required to respond using a strict format — `Level:<value>`, where `<value>` is a single integer between 1 and 4, ensuring consistency and ease of parsing.

The human-curated prompt design consists of three progressively richer variants:

- **Prompt A: Zero-shot Basic Instruction Prompt.** Inspired by [Loke et al. \(2025\)](#), this version gives the model only a concise instruction and a role assignment (linguist). It evaluates the model’s ability to perform zero-shot classification based purely on its pretrained understanding.
- **Prompt B: Zero-shot Instruction + Label Definitions.** To reduce ambiguity in classification boundaries, we extend Prompt A by explicitly adding the natural language descriptions of all four grammaticalization levels (found in [subsection 3.1](#)). This addition is motivated by [Münker et al. \(2024\)](#), who show that enriching prompts with label semantics significantly improves performance on German NLP tasks.
- **Prompt C: Few-shot with Interleaved Examples.** This prompt extends Prompt B with eight labeled examples (two per class) arranged in an interleaved order (1a, 2a, 3a, 4a, 1b, 2b, 3b, 4b) to reduce label-position bias and support generalization ([Lu et al., 2022](#)).

In all the prompts, each level description and basic task instruction remains consistent across prompt variants to allow meaningful comparison.

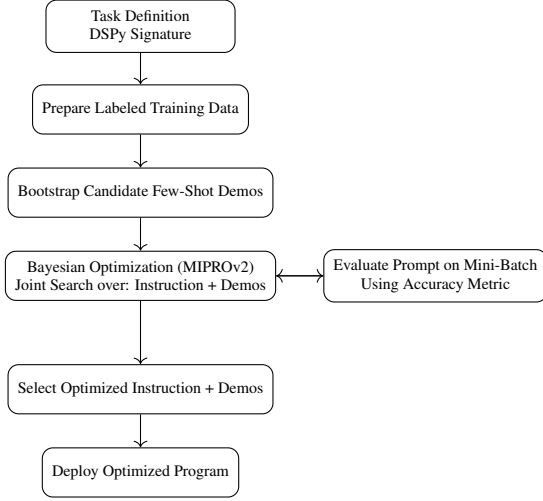


Figure 2: MIPROv2 optimization pipeline for jointly refining instructions and few-shot examples.

DSPy-Based Prompt Optimization

To scale beyond manual prompt engineering, we employ **DSPy** (Khattab et al., 2023), a declarative framework for LM program synthesis. DSPy allows us to define tasks modularly in the form of a signature (representation of input and output fields) and use optimizers to automatically discover effective prompt instructions and demonstrations.

We evaluate three DSPy-based prompt configurations:

- **Baseline DSPy (No Optimization):** A structured implementation of Prompt B using DSPy syntax, serving as a controlled comparison to human-curated prompts.
- **Zero-shot Optimized:** Using MIPROv2 Opsahl-Ong et al. (2024), we optimize only the instructional string while holding the label definitions fixed. This allows the optimizer to refine how the task is phrased without relying on examples.
- **Few-shot Optimized (Joint):** Here, MIPROv2 performs a joint optimization over both the instruction and the selection of few-shot examples. The optimizer bootstraps a candidate pool and uses Bayesian search to maximize performance on a held-out metric (accuracy).

A schematic overview of the joint optimization process is illustrated in Figure 2.

4.3 Leakage Check Strategy

The leakage check framework is built as a modular Python toolkit, supporting robust dataset loading, model interfacing, multiple probing methods, and unified reporting. The workflow consists of:

Leakage Probing Methods:

- **Method 1 a: Partial Recall.** Prompt with a masked cue (e.g., [x_...]) and a short instruction; count recovery if the gold token appears (Carlini et al., 2021; Hayes et al., 2025).
- **Method 1 b: ICL Recall.** Variant of Method 1 with k in context examples before the masked item to test context-triggered memorization (Carlini et al., 2021; Yasunaga et al., 2024).
- **Method 2: Real vs. Fake.** Run the Method 1 prompt on real expressions and synthetic German-like non-words; report the real-fake gap ($p_{\text{real}} - p_{\text{fake}}$); a large gap implies inventory knowledge beyond morphology (Wang et al., 2025; Xie et al., 2025).
- **Method 3: Half-Set Context.** Provide half the gold list as context and test recall on the rest; elevated recall signals sensitivity to exposed inventory (indirect leakage) (Rosenblatt et al., 2025).
- **Method 4: Next-in-Score.** Show a few items from score s and ask for “another”; count exact matches against the gold vocabulary to probe associative recall (Karamolegkou et al., 2023).
- **Method 5: Predict Score.** Ask for the 1 to 4 label; treat as leakage only if accuracy $\gg 25\%$ which would be random and co-occurs with recall signals (high Method 1 or large Method 2 gap); otherwise it may reflect generalization/priors (Chang et al., 2023).

Metric Aggregation:

We compute per-method scores and exported plots:

- **Recovery** (Method 1/ICL/Half): $\frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{gold}_i \subseteq \text{out}_i]$.
- **Real-fake gap** (Method 2): $\hat{p}_{\text{real}} - \hat{p}_{\text{fake}}$.
- **Match** (Method 4): $\frac{1}{B} \sum_{b=1}^B \mathbf{1}[\text{pred}_b \in V_s]$.
- **Accuracy** (Method 5): $\frac{1}{M} \sum_{i=1}^M \mathbf{1}[\hat{y}_i = y_i]$ (chance 25%).

Evidence of Leakage.

We consider leakage when at least one holds: (i) Method 1 or Method 2 $\geq 30\%$ (with ICL $\geq$ Method

1), (ii) Method 2 real-fake gap ≥ 20 points, or (iii) Method 5 accuracy $\geq 40\%$ together with (i) or (ii). This is a comparison based metric, the rates are based on our two potentially leaked German datasets; performance well above chance under partial masking (no semantic cues) and, especially, a large real-fake gap further support leakage.

5 Experiments

The whole code can be found on GitHub³.

5.1 Metric-Based Baselines

We needed to evaluate the predictive power of individual corpus-based metrics in order to compare these approaches with LLM predictions directly. To convert continuous metric values into discrete grammaticalization level predictions, we implemented a percentile-based threshold search procedure. For each metric, we first computed its Spearman correlation with the annotated grammaticalization scores to determine the directionality of the mapping (i.e., whether higher values correspond to higher or lower levels). Based on this, we searched for the best combination of three thresholds (t_1, t_2, t_3) separating the four levels by exhaustively evaluating all valid combinations from a fixed set of candidate percentiles (10th to 90th). For each candidate combination, metric values were mapped to predicted levels based on the threshold configuration, and accuracy was measured against gold labels. The final thresholds were those that yielded the highest classification accuracy on the development set.

In addition to the individual metrics, we included two simple baselines:

- **Random baseline:** Each expression is assigned a random level between 1 and 4.
- **Most frequent class baseline:** All predictions are fixed to level 3, the most frequent class in the dataset (see Figure 1).

Performance was evaluated on Testset using the **macro-averaged F1** score to account for class imbalance. The results in Figure 3 show that *Syntactic Context Adversity* achieved the highest score, outperforming the traditional *expression Frequency* metric, which was previously the strongest baseline. Both baselines performed the worst.

³github.com/CastIehard/LLM-4-gram-lvl-pred

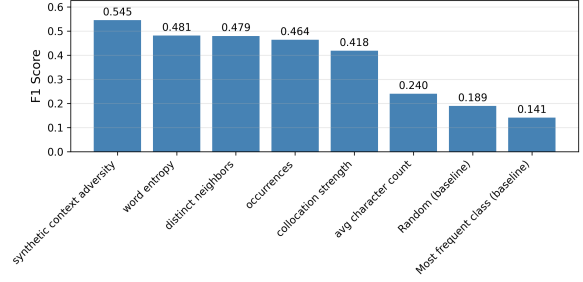


Figure 3: F1 scores for each metric of the test set.

Additionally, to facilitate direct comparison with Schlechtweg and Walde (2018), we include a table reporting each metric together with its Spearman correlation, enabling a one-to-one comparison with their results (Table 5).

5.2 Prompting experimental setup

We conduct two main sets of experiments to assess the impact of various prompting strategies on grammaticalization classification performance for both models GPT-4o-mini and GPT-4o.

Stage 1 – Manual Prompt Variants. We test three progressively enriched prompt designs, each sharing the same core instruction. Prompt A contains minimal label descriptions, while Prompt B and Prompt C both incorporate detailed label definitions, as shown in Figure 9. Adding label definitions leads to a clear performance improvement, so we consider Prompt B (with label descriptions) as our new baseline. Prompt C extends this by including few-shot examples in addition to the label definitions, and is expected to yield even better results.

Stage 2 – DSPy Module Variants. To directly compare manual prompting with a programmable interface, we utilize the DSPy framework. The task is modeled with `preposition` and `label definition` as input fields, and `level` as the output field. We began with a plain DSPy module (non-optimized), then applied the MIPROv2 optimizer to tune the instructions based on training examples. Clear instruction tuning can be seen from figure 7 and figure 8 in the appendix. Due to limited data, we used 32 examples, which is 8 per level. Finally, we test the joint optimization setting in which both the instruction and the example set were selected automatically. The optimizer is trained using accuracy as the evaluation metric.

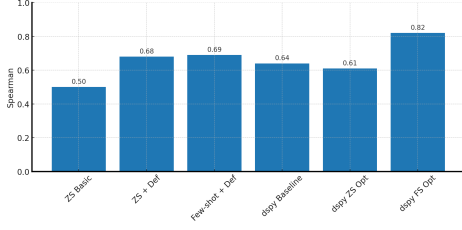


Figure 4: Spearman Correlation for GPT-4o across prompting strategies

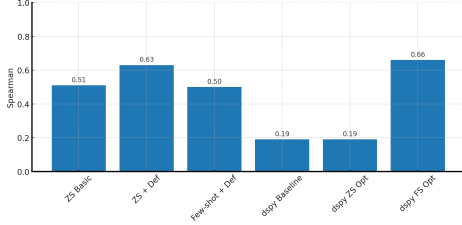


Figure 5: Spearman Correlation for GPT-4o-mini across prompting strategies

5.3 Result Analysis

To evaluate the grammaticalization degree prediction performance, we analyzed the models across three metrics: Average Precision (AP) and Spearman Correlation. The results highlight substantial differences across prompting strategies and model variants (GPT-4o vs GPT-4o-mini).

Spearman Correlation Analysis

Figure 4 and Figure 5 illustrate the Spearman correlation between the predicted and true grammaticalization degrees for both GPT-4o and GPT-4o-mini.

From the plots, we observe the following:

- As expected, for **GPT-4o**, the DSPy few-shot Optimization strategy achieves the highest Spearman correlation of 0.82, indicating a strong monotonic relationship with true labels.
- Among human-curated prompting methods, Zero-Shot and Few-shot both with level description perform well (0.68 and 0.69, respectively), showing that definition-enhanced strategies help generalization.
- **GPT-4o-mini**, being a smaller model, shows lower correlations overall. However, it still benefits significantly from Zero-shot with level description (0.63) and DSPy few-shot Optimization strategy (0.66), confirming that these methods are consistently helpful across model sizes.

- The DSPy Baseline(non-optimized) and DSPy Zero-shot optimized for GPT-4o-mini yield very low Spearman values (0.19), indicating less reliability for ordinal predictions under these settings.

Average Precision Comparison We evaluated the discriminative power of the models using Average Precision (AP) for binary class comparisons between degrees of grammaticalization. Table 3 (manual prompting) shows that the pair $AP(2 \text{ vs. } 4)$ is the most separable: GPT-4o reaches **1.00** with few-shot (FS.def), while GPT-4o-mini peaks at **0.95** with the optimized zero-shot prompt (ZS.def). In contrast, $AP(1 \text{ vs. } 2)$ is the hardest pair (best scores only **0.41** for GPT-4o-mini with ZS, and **0.37** for GPT-4o with ZS.def). For $AP(1 \text{ vs. } 3)$, GPT-4o performs best with few-shot (**0.70**), whereas GPT-4o-mini benefits more from ZS.def (**0.67**).

Table 4 (DSPy-based prompting) further confirms these trends: few-shot optimization (FS.opt) lifts GPT-4o on the difficult $AP(1 \text{ vs. } 2)$ pair (up to **0.56**) and achieves a perfect **1.00** on the easier $AP(2 \text{ vs. } 4)$ pair. Overall, the results indicate that (i) higher grammaticalization gaps (2 vs. 4) yield near-perfect separability, (ii) adjacent levels (1 vs. 2) remain challenging, and (iii) the best strategy depends on the model—GPT-4o tends to favor few-shot, while GPT-4o-mini often benefits from optimized zero-shot prompts.

5.4 Leakage Check Experiments

Experimental Setup We evaluate original and two candidate-leak splits (see subsection 3.4), apply six probes (subsection 4.3), and save per-item CSVs plus summary metrics. Models: primary GPT-4o; decoding is deterministic (temperature 0). The leakage probing template is given in Figure 10.

Results. Figure 6, Figure 11 and Figure 12 summarize per-method scores for GPT-4o.

Findings. On `c4de/wiki`, recall is high (Method 1 $\approx 44\text{--}50\%$), increases with ICL/Half ($\approx 44\text{--}50\%$), and Method 2 shows a large real-fake gap (50.00% vs. 0–0.39%), indicating exposure to the true inventory. On `original`, recall is low (Method 1=9.22%, ICL=7.48%) while Method 5 is higher (34.02%), consistent with priors rather than memorisation. We therefore treat `c4de/wiki` as *operationally contaminated* and use `original` for primary evaluation, reporting all probe metrics;

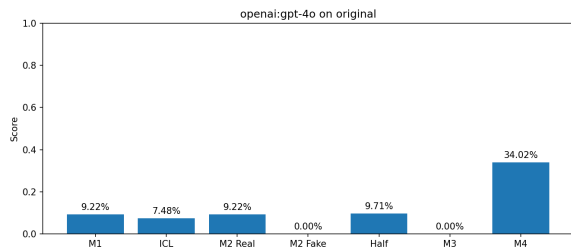


Figure 6: GPT-4o on `original`. Partial-recall rates remain low (Method 1=9.22%, ICL=7.48%), Half-Set recall is modest (9.71%), while Method 5 label prediction is comparatively higher (34.02%).

training inclusion remains unverified (closed corpora undisclosed; no full audit for open).

6 Discussion

Our experiments have shown that both traditional metric-based baselines and LLM-based prompting approaches can provide useful signals for predicting grammaticalization levels. Performance varies across levels and approaches, indicating that further optimization is required.

One promising research direction is to evaluate a broader range of LLM architectures and sizes to assess whether larger or more specialized models yield better results. Our findings show that including explicit level definitions in prompts significantly improves performance, emphasizing the importance of clear, linguistically grounded instructions. While joint optimization of instructions and examples (via DSPy) performed well, it was trained on only 32 examples and evaluated using simple accuracy, leaving room for improved metrics and more robust optimization. Interestingly, in both GPT-4o and GPT-4o-mini, zero-shot optimization underperformed even compared to manually designed prompts, highlighting the need to explore better optimizers. Moreover, in GPT-4o-mini, few-shot prompting lagged behind definition-based zero-shot prompting, suggesting further opportunities to refine prompting strategies through more scalable and adaptive methods.

Our “contaminated” tag for `c4de/wiki` is based on probe signals without formal membership testing. The current heuristic thresholds require calibration against both clean and known-leak datasets. Since these probes rely on comparisons (e.g., real vs. fake), they may conflate genuine generalization with leakage. Future work should focus on validating these probes across confirmed leak datasets to

establish robust and transferable thresholds.

7 Conclusion

We demonstrated that LLMs can predict grammaticalization levels more effectively than traditional corpus-based metrics. We further showed that LLMs possess a strong inherent ability to model grammaticalization phenomena and that including explicit definitions within prompts significantly boosts their predictive performance.

Our investigation also revealed that strict output formatting is essential for ensuring consistency and facilitating automated evaluation, and that debugging such systems, while challenging, is achievable through careful analysis.

In parallel, we conducted leakage checks using partial-recall, in-context, real vs fake, and half-set probes under deterministic decoding. Strong contamination signals were found in the `c4de` and `wiki` splits, while `original` showed weak recall but some label prediction. As a result, we consider `c4de/wiki` contaminated and use `original` for main reporting, providing probe metrics to help interpret potential leakage.

Now that we have identified the best-performing prompt approach, established the importance of definitions, and clarified the methodological choices that maximize performance by adding examples in a specific ordering and class-balanced manner, these insights can be applied at scale. In the future, we can try the same pipeline on a much bigger corpus and try different prompting techniques and optimizers, including weighted few-shot. The framework could be extended to predict grammaticalization levels for a much larger set of lexical items, enabling large-scale analysis of grammaticalization patterns across corpora. This opens the door to broader linguistic studies, comparative analyses across languages, and integration into larger natural language processing pipelines.

References

- Simone Balloccu, Patrícia Schmidtová, Mateusz Lango, and Ondřej Dušek. 2024. [Leak, cheat, repeat: Data contamination and evaluation malpractices in closed-source llms](#).
- Marco Baroni, Silvia Bernardini, Adriano Ferraresi, and Eros Zanchetta. 2009. [The WaCky wide web: a collection of very large linguistically processed web-crawled corpora](#). *Language Resources and Evaluation*, 43(3):209–226.

- Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwal, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. 2023. [Extracting training data from diffusion models](#).
- Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. [Extracting training data from large language models](#).
- Kent K. Chang, Mackenzie Cramer, Sandeep Soni, and David Bamman. 2023. [Speak, memory: An archaeology of books known to chatgpt/gpt-4](#).
- David Correia Saavedra. 2019. *Measurements of grammaticalization : developing a quantitative index for the study of grammatical change*. Doctorate, Université de Neuchâtel, Neuchâtel, Switzerland.
- Claudio Di Meola. 2014. *Die Grammatikalisierung deutscher Präpositionen*, 2., aktualisierte Aufl. 2014 der 1. Aufl. edition. Number Bd. 62 in Studien zur deutschen Grammatik. Stauffenburg-Verl, Tübingen.
- Shahriar Golchin and Mihai Surdeanu. 2024. [Time travel in llms: Tracing data contamination in large language models](#).
- Jamie Hayes, Marika Swanberg, Harsh Chaudhari, Itay Yona, Ilia Shumailov, Milad Nasr, Christopher A. Choquette-Choo, Katherine Lee, and A. Feder Cooper. 2025. [Measuring memorization in language models via probabilistic extraction](#).
- Paul J. Hopper and Elizabeth Closs Traugott. 2003. *Grammaticalization*, 2nd ed edition. Cambridge textbooks in linguistics. Cambridge University Press, Cambridge, UK New York, NY.
- Changwei Hu, Yu Zhu, and Liangjie Yuan. 2025. [Influences of Language Functions on Linguistic Features: Multi-Dimensional and Entropy Analyses of Academic and Entertainment Registers](#). *Entropy*, 27(8):783.
- Antonia Karamolegkou, Jiaang Li, Li Zhou, and Anders Søgaard. 2023. [Copyright violations and large language models](#).
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. [Dspy: Compiling declarative language model calls into self-improving pipelines](#).
- Ying Xuan Loke, Dominik Schlechtweg, and Wei Zhao. 2025. [ABDN-NLP at CoMeDi shared task: Predicting the aggregated human judgment via weighted few-shot prompting](#). In *Proceedings of Context and Meaning: Navigating Disagreements in NLP Annotation*, pages 122–128, Abu Dhabi, UAE. International Committee on Computational Linguistics.
- Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. [Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity](#).
- Claudio Di Meola. 2012. [Deutsche Präpositionen im Überblick: Form, Stellung und Rektion](#). *Pan-daemonium Germanicum : revista de estudos germanísticos*, 4(1):321 – 368.
- Simon Münker, Kai Kugler, and Achim Rettinger. 2024. [Zero-shot prompt-based classification: topic labeling in times of foundation models in german tweets](#).
- Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. [Optimizing instructions and demonstrations for multi-stage language model programs](#).
- Lucas Rosenblatt, Bin Han, Robert Wolfe, and Bill Howe. 2025. [Fragments to facts: Partial-information fragment inference from llms](#).
- Dominik Schlechtweg and Sabine Schulte im Walde. 2018. [Distribution-based Prediction of the Degree of Grammaticalization for German Prepositions](#). ArXiv:1804.06719 [cs].
- Alejandro A. Torres-García, Omar Mendoza-Montoya, Marta Molinas, Javier M. Antelis, Luis A. Moctezuma, and Tonatiuh Hernández-Del-Toro. 2022. [Chapter 4 - pre-processing and feature extraction](#). In Alejandro A. Torres-García, Carlos A. Reyes-García, Luis Villaseñor-Pineda, and Omar Mendoza-Montoya, editors, *Biosignal Processing and Classification Using Computational Learning and Intelligence*, pages 59–91. Academic Press.
- Thomas Walshe, Sae Young Moon, Chunyang Xiao, Yawwani Gunawardana, and Fran Silavong. 2025. [Automatic labelling with open-source llms using dynamic label schema integration](#).
- Cheng Wang, Yiwei Wang, Bryan Hooi, Yujun Cai, Nanyun Peng, and Kai-Wei Chang. 2025. [Con-recall: Detecting pre-training data in llms via contrastive decoding](#).
- Roy Xie, Junlin Wang, Ruomin Huang, Minking Zhang, Rong Ge, Jian Pei, Neil Zhenqiang Gong, and Bhuwan Dhingra. 2025. [Recall: Membership inference via relative conditional log-likelihoods](#).
- Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H. Chi, and Denny Zhou. 2024. [Large language models as analogical reasoners](#).
- Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. [Calibrate before use: Improving few-shot performance of language models](#).

Appendices

A Who did what?

Prasanna was responsible for all work related to prompting. This included the complete research on both classical and advanced prompting methods, including few-shot prompting. In addition, Prasanna developed and carried out the approach using the DSPY framework.

Divyansh handled all aspects related to leakage checks. He conducted research on potential leakage scenarios, developed testing procedures, and ensured that the experiments were protected against unintended information leaks.

Luca was responsible for all data preparation tasks, including pre-processing the data and calculating all metrics needed to compare to [Schlechtweg and Walde \(2018\)](#) paper. Furthermore he added additional metrics he found in related papers. He also developed the evaluation pipeline and approach to comparing all the different approaches with each other, using a predefined file structure for outputs to enable the creation of all plots. Due to the limitations of local LLMs, Luca used his GPT-4o access to run the pipelines. As Luca was the only German speaker in the team, he also reviewed all the foundational papers in German, extracting and formulating the definitions for the grammaticalization levels. Furthermore, he was responsible for all content in the document that did not directly relate to prompting or leakage checks. This included structuring the general sections, drafting foundational content and producing supporting analyses.

It is important to emphasize that each group member is individually responsible for their own work. While we reviewed one another’s contributions, accountability for cited sources and the use of AI tools rests solely with the respective author. Prasanna for everything related to prompting, Divyansh for everything related to leakage and Luca for everything else.

B AI Statement

In this work, we made extensive and deliberate use of AI tools to support both the presentation of ideas and the execution of technical tasks. Our primary development environment was Visual Studio Code, where we actively used GitHub Copilot⁴ in Agent Mode, as well as its integrated chat and

⁴github.com/features/copilot

auto-completion functionalities. These tools were particularly helpful for rapid prototyping, code suggestions, and drafting text sections.

For language refinement, we employed DeepL Write⁵ and related deep learning-based tools to rephrase sentences and improve stylistic clarity. Additionally, we used Anthropic’s Claude AI⁶ and ChatGPT⁷ for a range of tasks, including document formatting and summarizing relevant academic papers to accelerate our literature review process.

In the context of coding, AI proved to be extremely helpful for generating initial implementations and offering alternative approaches. However, we identified a significant concern: AI-generated code can be difficult to fully understand without careful review. While AI can produce syntactically correct and seemingly functional code, its structural soundness and logical consistency are not guaranteed. This requires the human developer to critically assess the generated output, ensuring it aligns with the intended design and problem requirements.

Overall, AI served as a powerful accelerator in our workflow, but we exercised caution to maintain full comprehension and ownership of the ideas, structure, and code that formed the basis of our project.

C Prompting and results

Prompt Before and After Optimization

Due to space constraints, we abstract the full label descriptions into a placeholder tag `<definitions>` in both examples below. The input structure remains unchanged, but the key difference lies in the optimized instructional phrasing after applying MIPROv2.

Model	AP Pair	zs	zs.def	fs.def
GPT-4o mini	AP (1 vs. 2)	0.41	0.37	0.38
	AP (1 vs. 3)	0.58	0.67	0.58
	AP (1 vs. 4)	0.80	0.88	0.76
	AP (2 vs. 4)	0.85	0.95	0.83
GPT-4o	AP (1 vs. 2)	0.31	0.37	0.34
	AP (1 vs. 3)	0.63	0.61	0.70
	AP (1 vs. 4)	0.43	0.97	1.00
	AP (2 vs. 4)	0.59	0.98	1.00

Table 3: Average Precision (AP) scores for grammaticalization degree comparisons using Zero-shot (ZS) and Few-shot (FS) for manual prompting strategies.

⁵<https://www.deepl.com/en/write>

⁶claude.ai

⁷chatgpt.com

Prompt Template Before Optimization
system: Your input fields are: 1. label_definitions (str): Text describing label meanings 1--4 2. preposition (str): The German preposition Your output fields are: 1. level (str): Output in format: Level : <value> All interactions will be structured in the following way, with the appropriate values filled in. <pre>[[## label_definitions ##]] {label_definitions}</pre> <pre>[[## preposition ##]] {preposition}</pre> Outputs will be a JSON object with the following fields: <pre>{ "level": "{level}" }</pre> In adhering to this structure, your objective is: You are a highly trained linguist specializing in German grammaticalization and capable of providing subjective responses. Rate the degree of grammaticalization of the target German word by utilizing the following level description -- label between *’s followed by its definition. Your response should align with a human’s succinct judgment. Please respond in the format: Level : <value> user: [[## label_definitions ##]] <Definition> <pre>[[## preposition ##]] in</pre> Respond with a JSON object in the following order of fields: level.

Figure 7: Prompt template before any optimization.

D Spearman Correlation Grammaticalization Level and Metric

Legend for columns A–F in table [Table 5](#):

1. frequency
2. word entropy
3. distinct neighbors
4. collocation strength
5. syntactic context adversity
6. average character count

Prompt Template After Optimization (MIPROv2)
system: Your input fields are: 1. label_definitions (str): Text describing label meanings 1--4 2. preposition (str): The German preposition Your output fields are: 1. level (str): Output in format: Level: <value> All interactions will be structured in the following way, with appropriate values filled in: <pre>[[## label_definitions ##]] <definitions></pre> <pre>[[## preposition ##]] in</pre> Outputs will be JSON object with the following: <pre>{ "level": "<value>" }</pre> In adhering to this structure, your objective is: You are a highly trained linguist specializing in German grammaticalization and capable of providing subjective responses. Rate the degree of grammaticalization of the target German word by utilizing the following level description label between *’s followed by its definition. Your response should align with a human’s succinct judgment. Please respond in the format: Level : <value> Respond with a JSON object in the following order of fields: level.

Figure 8: Prompt template after optimization using MIPROv2.

Model	AP Pair	zs	zs.opt	fs.opt
GPT-4o mini	AP (1 vs. 2)	0.41	0.41	0.40
	AP (1 vs. 3)	0.68	0.68	0.58
	AP (1 vs. 4)	0.39	0.39	1.00
	AP (2 vs. 4)	0.68	0.68	1.00
GPT-4o	AP (1 vs. 2)	0.34	0.32	0.56
	AP (1 vs. 3)	0.67	0.70	0.74
	AP (1 vs. 4)	0.90	0.86	1.00
	AP (2 vs. 4)	0.93	0.90	1.00

Table 4: Average Precision (AP) scores for grammaticalization degree comparisons using DSPy-based prompting strategies.

Evaluation	A	B	C	D	E	F
Spearman’s ρ (rank)	0.54	0.42	0.53	-0.43	0.56	0.23
AP (degrees 1 vs. 2)	0.58	0.45	0.57	0.32	0.60	0.38
AP (degrees 1 vs. 3)	0.69	0.62	0.68	0.51	0.73	0.67
AP (degrees 1 vs. 4)	0.98	0.95	0.99	0.30	0.98	0.57
AP (degrees 2 vs. 3)	0.71	0.72	0.70	0.71	0.70	0.78
AP (degrees 2 vs. 4)	0.98	0.95	0.98	0.44	0.98	0.71
AP (degrees 3 vs. 4)	0.96	0.93	0.98	0.26	0.96	0.41

Table 5: Results for metric-based prediction

Basic Prompt Template

[SYSTEM] You are a highly trained linguist specializing in German grammaticalization and capable of providing subjective responses. Rate the degree of grammaticalization of the target German word by utilizing the following level description – label between *’s followed by its definition: DEFINITIONS
Your response should align with a human’s succinct judgment. Please respond in the format: Level : ;value;
[USER] Keyword (target word): ;keyword; Please provide a judgment at a single level. For example, if your judgment is fully grammaticalized, then provide Level: 4. If your judgment is very low grammaticalization, provide Level: 1.

Figure 9: Basic prompt template with level definitions used for grammaticalization degree prediction experiments.

Leakage Probing Templates (M1–M4, ICL, Half-Set)

Prompt templates. Below we list the exact templates (abbreviated from core content) used in the experiments.

1. **M1 Partial recall:**
Fill in the missing German preposition: '[x____]'.
2. **ICL Partial recall with context:**
Here are German prepositions: [kw_1, ..., kw_k]. Now complete: '[x____]'.
3. **M2 Real vs. Fake:**
Complete the German preposition: '[x____]'.
(Run on real keywords and on synthetic German-like non-words.)
4. **Half-Set context:**
Below is a list of German prepositions (context): [kw_1, ...]. Now complete: '[x____]'.
5. **M3 Next-in-Score:**
Here are German prepositions with grammaticalization score [s]: [kw_1, ...]. Suggest another from the same score:
6. **M4 Predict score:**
What is the grammaticalization score (1=very low, 4=very high) of the German preposition '[kw]'? Respond with one digit 1--4.

Figure 10: Leakage probing prompt templates (M1–M4, ICL, Half-Set).

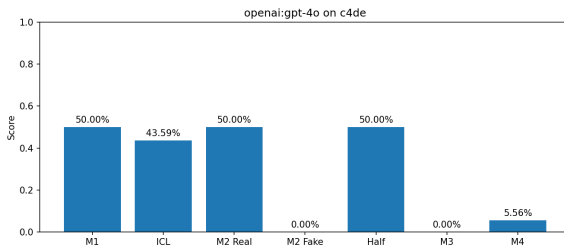


Figure 11: gpt-4o on c4de. Notable: high partial recall (M1=50.00%), strong ICL (43.59%), real $\gg$ fake in M2 (50.00% vs. 0.00%), and robust Half-Set recall (50.00%).

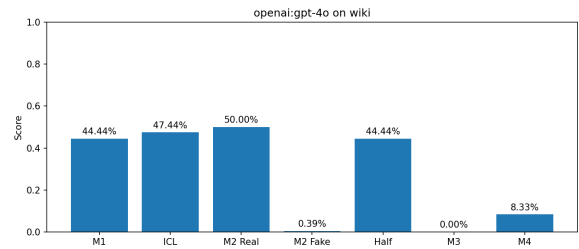


Figure 12: gpt-4o on wiki. Elevated partial recall (M1=44.44%, ICL=47.44%), strong real $\gg$ fake gap (M2: 50.00% vs. 0.39%), and high Half-Set recall (44.44%).